SEMANA 5

ACTIVIDAD 1

Reto 1: Proyecto — Sistema de Edición CodeLeap con Pila propia

Archivo de código con: (1) Clase Nodo y Clase Pila completamente implementadas con comentarios explicando cada línea clave, (2) Clase EditorCodeLeap con métodos escribir, deshacer, rehacer y mostrarHistorial, (3) Escenario de prueba del enunciado ejecutado con salida visible, (4) Comentario al inicio explicando con tus palabras por qué se necesitan dos pilas para el sistema deshacer/rehacer.

```
#include <iostream>

#include <string>

using namespace std;

class Nodo {

public:

   string dato;

   Nodo* siguiente;

   Nodo(string valor) {

      dato = valor;

      siguiente = nullptr;

   }

};

class Pila {

private:

   Nodo* tope;

   int contador;

public:

   Pila() {

      tope = nullptr;

      contador = 0;

   }

   void push(string valor) {
```

```
    Nodo* nuevo = new Nodo(valor);
    nuevo->siguiente = tope;
    tope = nuevo;
    contador++;
}
string pop() {
    if (estaVacia()) {
        return "";
    }
    Nodo* nodoRetirado = tope;
    string valor = nodoRetirado->dato;
    tope = nodoRetirado->siguiente;
    contador--;
    delete nodoRetirado;
    return valor;
}
string peek() {
    if (estaVacia()) {
        return "";
    }
    return tope->dato;
}
bool estaVacia() {
    return contador == 0;
}
int tamaño() {
    return contador;
}
void mostrar() {
```

```
        Nodo* actual = tope;
        cout << "[";
        while (actual != nullptr) {
            cout << "\"" << actual->dato << "\"";
            if (actual->siguiente != nullptr) cout << ", ";
            actual = actual->siguiente;
        }
        cout << "]" << endl;
    }
};
class EditorCodeLeap {
private:
    Pila pilaDeshacer;
    Pila pilaRehacer;
public:
    void escribir(string cambio) {
        pilaDeshacer.push(cambio);
        pilaRehacer = Pila();
    }
    void deshacer() {
        if (!pilaDeshacer.estaVacia()) {
            string cambio = pilaDeshacer.pop();
            pilaRehacer.push(cambio);
        }
    }
    void rehacer() {
        if (!pilaRehacer.estaVacia()) {
            string cambio = pilaRehacer.pop();
            pilaDeshacer.push(cambio);
```

```
        }
    }
    void mostrarHistorial() {
        cout << "Historial Deshacer (pilaDeshacer): ";
        pilaDeshacer.mostrar();
        cout << "Historial Rehacer (pilaRehacer): ";
        pilaRehacer.mostrar();
    }
};
int main() {
    EditorCodeLeap editor;
    editor.escribir("def funcion():");
    editor.escribir("    x = 5");
    editor.escribir("    return x");
    editor.deshacer();
    editor.deshacer();
    editor.rehacer();
    editor.mostrarHistorial();
    return 0;
}
```

SALIDA ESPERADA DE PRUEBA EN C++

```
Historial Deshacer (pilaDeshacer): ["def funcion():", "    x = 5"]
Historial Rehacer (pilaRehacer): ["    return x"]
```

ACTIVIDAD 2

Reto 2: Verificador de Paréntesis para el Linter de CodeLeap

Método verificarBalanceo() integrado en la clase EditorCodeLeap, con las 5 pruebas del enunciado ejecutadas y sus resultados. Incluye un comentario personal de 3-4 líneas explicando por qué una pila es esencial aquí y un simple contador no funcionaría.

```
#include <iostream>
#include <string>
using namespace std;
class Nodo {
public:
   string dato;
   Nodo* siguiente;
   Nodo(string valor) {
      dato = valor;
      siguiente = nullptr;
   }
};
class Pila {
private:
   Nodo* tope;
   int contador;
public:
   Pila() {
      tope = nullptr;
      contador = 0;
   }
   void push(string valor) {
      Nodo* nuevo = new Nodo(valor);
      nuevo->siguiente = tope;
      tope = nuevo;
      contador++;
   }
   string pop() {
      if (estaVacia()) return "";
      Nodo* nodoRetirado = tope;
      string valor = nodoRetirado->dato;
      tope = nodoRetirado->siguiente;
      contador--;
      delete nodoRetirado;
      return valor;
   }
```

```
    string peek() {
        if (estaVacia()) return "";
        return tope->dato;
    }
    bool estaVacia() {
        return contador == 0;
    }
    int tamaño() {
        return contador;
    }
    void mostrar() {
        Nodo* actual = tope;
        cout << "[";
        while (actual != nullptr) {
            cout << "\"" << actual->dato << "\"";
            if (actual->siguiente != nullptr) cout << ", ";
            actual = actual->siguiente;
        }
        cout << "]" << endl;
    }
};
class EditorCodeLeap {
private:
    Pila pilaDeshacer;
    Pila pilaRehacer;
public:
    void escribir(string cambio) {
        pilaDeshacer.push(cambio);
        pilaRehacer = Pila();
    }
    void deshacer() {
        if (!pilaDeshacer.estaVacia()) {
            string cambio = pilaDeshacer.pop();
            pilaRehacer.push(cambio);
        }
    }
    void rehacer() {
        if (!pilaRehacer.estaVacia()) {
            string cambio = pilaRehacer.pop();
            pilaDeshacer.push(cambio);
        }
    }
    void mostrarHistorial() {
        cout << "Historial Deshacer (pilaDeshacer): ";
```

```
        pilaDeshacer.mostrar();
        cout << "Historial Rehacer (pilaRehacer): ";
        pilaRehacer.mostrar();
    }
    pair<bool,string> verificarBalanceo(string codigoFuente) {
        Pila pila;
        for (char c : codigoFuente) {
            if (c == '(' || c == '[' || c == '{') {
                pila.push(string(1,c));
            } else if (c == ')' || c == ']' || c == '}') {
                if (pila.estaVacia()) {
                    return {false, "Cierre sin apertura"};
                }
                string tope = pila.pop();
                if ((c == ')' && tope != "(") ||
                    (c == ']' && tope != "[") ||
                    (c == '}' && tope != "{")) {
                    return {false, "Bracket incorrecto al cerrar"};
                }
            }
        }
        if (!pila.estaVacia()) {
            return {false, "Faltan cierres"};
        }
        return {true, "Código balanceado"};
    }
};
int main() {
    EditorCodeLeap editor;

    string pruebas[] = {"({}[])", "({[]})", "({[}])", "((((", ")))"};
    for (string prueba : pruebas) {
        auto resultado = editor.verificarBalanceo(prueba);
        cout << "Prueba: " << prueba << " -> "
             << (resultado.first ? "VÁLIDO" : "INVÁLIDO")
             << " (" << resultado.second << ")" << endl;
    }

    return 0;
}
```

RESULTADO DE LAS 5 PRUEBAS EN C++

Prueba: ({}[]) -> VÁLIDO (Código balanceado)
Prueba: ({[]}) -> VÁLIDO (Código balanceado)
Prueba: ({[}]) -> INVÁLIDO (Bracket incorrecto al cerrar)
Prueba: (((-> INVÁLIDO (Faltan cierres)
Prueba:))) -> INVÁLIDO (Cierre sin apertura)

ACTIVIDAD 3

Reto 3: Trade-offs y Decisiones Técnicas

Documento escrito (no código) de media página con: (1) Tu posición final sobre lista enlazada vs arreglo para pilas, justificada, (2) La decisión que tomarías para el historial del editor y por qué, (3) Un criterio personal para elegir cuándo usar pila vs otra estructura.

1. Lista enlazada vs arreglo dinámico para pilas Mi posición final es que, aunque la lista enlazada ofrece flexibilidad teórica al no tener un límite fijo, en la práctica un arreglo dinámico suele ser más eficiente. Los arreglos aprovechan mejor la memoria porque no requieren punteros adicionales y permiten acceso directo a cualquier posición. Además, las bibliotecas modernas ya implementan redimensionamiento automático, lo que elimina la preocupación por un límite rígido. La lista enlazada sigue siendo útil en contextos académicos o cuando las operaciones de inserción/eliminación son muy frecuentes y el tamaño es impredecible, pero en producción el arreglo dinámico ofrece mejor rendimiento general.

2. Decisión para el historial del editor En mi EditorCodeLeap, optaría por guardar el texto completo en cada paso. Aunque esto consume más memoria, es mucho más simple de implementar y facilita el proceso de deshacer/rehacer sin necesidad de reconstruir el documento a partir de múltiples cambios. Para un proyecto educativo, la claridad y la facilidad de comprensión pesan más que la optimización. En un editor profesional como VS Code, sí se usaría una estrategia híbrida: guardar deltas para eficiencia y snapshots completos cada cierto tiempo para evitar aplicar miles de cambios uno por uno.

3. Criterio personal para elegir cuándo usar pila vs otra estructura Usaré una pila cuando el problema requiera un orden LIFO natural: lo último que entra es lo primero que debe salir, como en el sistema de deshacer o en la navegación “Atrás” del navegador. Si el problema requiere acceso aleatorio, orden FIFO, o relaciones más complejas, entonces consideraré otras estructuras como arreglos, colas o árboles. Mi criterio personal es preguntarme: *¿la lógica del problema se comporta como una torre de papeles donde siempre retiro el último que puse?* Si la respuesta es sí, la pila es la estructura correcta; si no, debo buscar otra alternativa.